



SISTEMA DE BIBLIOTECA

Projeto Integrador — Entrega Parcial 4

CRUD Completo + Validações + Mensagens

Aluno: Hezron Daniel M C Canturil | Leandro Caitano | Luilson Vieira

Polo: Sobradinho — BA | Equipe: 03

github.com/luilsonvmartins/Projeto-e-Implementa-o-de-Sistemas-para-Web-2

PHP 8+

MySQL

MVC

PDO

CRUD

XAMPP

Identificação	
Alunos	Hezron Daniel M C Canturil Leandro Caitano dos Santos Luilson Vieira Martins
Polo	Sobradinho — BA
Disciplina	Projeto e Implementação de Sistemas para Web II
Entrega	Entrega Parcial 4 — Semana 9 (60 pts) — CRUD Completo
Repositório	github.com/luilsonvmartins/Projeto-e-Implementa-o-de-Sistemas-para-Web-2

1. Objetivo da Entrega

A Entrega Parcial 4 conclui o gerenciamento das entidades principais do sistema, adicionando as operações de **Update (edição)** e **Delete (remoção)** com **validações completas** nos formulários e **mensagens de sucesso e erro** em todas as operações CRUD. O sistema agora é totalmente funcional para livros, autores e categorias.

2. Novidades em Relação à EP3

Funcionalidade	Descrição
Update — Livros	Formulário de edição pré-preenchido com dados do banco, atualiza título, ISBN, ano, sinopse, quantidade, categoria, autores e capa
Delete — Livros	Remoção com modal de confirmação, exclui também a imagem de capa do disco
Update — Autores	Edição de nome e nacionalidade com validação
Delete — Autores	Remoção com confirmação via modal
Update — Categorias	Edição de nome e descrição
Delete — Categorias	Remoção com confirmação
Validações server-side	Campos obrigatórios, tamanhos máximos, ano válido, tamanho de arquivo — erros exibidos no formulário sem perder os dados
Validação client-side	Atributos HTML5 (required, maxlength, min, max, accept) para feedback imediato
Modal de confirmação	JavaScript exibe modal antes de qualquer exclusão, evitando remoções acidentais
Mensagens flash	Sucesso (verde) e erro (vermelho) após cada operação, exibidos no topo da página
Upload com validação	Verifica extensão e tamanho (máx. 2MB) antes de salvar a capa

3. Update — Edição de Registros

O método **atualizar()** recebe os dados do formulário via POST, executa as validações e utiliza um **UPDATE com prepared statement** para persistir as alterações com segurança.

app/models/Livro.php — método atualizar()

```
public function atualizar(int $id, array $d): bool {
    $campos = 'titulo=:titulo,isbn=:isbn,ano=:ano,sinopse=:sinopse,',
              'qtd_total=:qtd,id_categoria=:cat';
    $params = [
        ':titulo' => trim($d['titulo']),
        ':isbn'   => trim($d['isbn'] ?? '') ?: null,
        ':ano'    => $d['ano'] ?: null,
        ':qtd'    => (int)$d['qtd_total'],
        ':cat'    => $d['id_categoria'] ?: null,
        ':id'     => $id,
    ];
    // Só atualiza capa se uma nova foi enviada
    if (!empty($d['capa'])) {
        $campos .= ',capa=:capa';
        $params[':capa'] = $d['capa'];
    }
    return $this->db->prepare(
        "UPDATE livros SET $campos WHERE id=:id"
    )->execute($params);
}
```

4. Delete — Remoção de Registros

A remoção de um livro também exclui a imagem de capa do servidor, evitando arquivos órfãos. Um **modal de confirmação** em JavaScript protege contra exclusões acidentais.

app/models/Livro.php — método deletar()

```
public function deletar(int $id): bool {
    // Remove imagem do disco antes de deletar o registro
    $stmt = $this->db->prepare('SELECT capa FROM livros WHERE id=?');
    $stmt->execute([$id]);
    $livro = $stmt->fetch();
    if ($livro && $livro['capa']) {
        $arquivo = ROOT . '/public/uploads/capas/' . $livro['capa'];
        if (file_exists($arquivo)) unlink($arquivo);
    }
    return $this->db->prepare('DELETE FROM livros WHERE id=?')->execute([$id]);
}
```

app/views/layout/header.php — modal de confirmação (JavaScript)

```
function confirmarExclusao(url, nome) {  
    document.getElementById('modalMensagem').innerHTML =  
        'Tem certeza que deseja remover <strong>' + nome + '</strong>?';  
    document.getElementById('modalBtnConfirmar').href = url;  
    document.getElementById('modalExcluir').classList.add('ativo');  
}  
// Uso nas views:  
// <button onclick="confirmarExclusao('url/deletar/1', 'Dom Casmurro')">
```

5. Validações Implementadas

As validações acontecem em **duas camadas**: no cliente (HTML5) para feedback imediato, e no servidor (PHP) para garantia de segurança e integridade.

Campo	Regra	Onde	
Título	Obrigatório, máx. 200 caracteres	<code>required + maxlength=200</code>	■
ISBN	Máx. 20 caracteres	<code>maxlength=20</code>	■
Ano	Entre 1000 e 2099	<code>min=1000 max=2099 + PHP</code>	■
Quantidade	Mínimo 1	<code>min=1 required + PHP</code>	■
Capa	JPG/PNG/GIF/WebP, máx. 2MB	<code>accept=image/* + PHP</code>	■
Nome (Autor)	Obrigatório, máx. 150 caracteres	<code>required + maxlength=150</code>	■
Nome (Categoria)	Obrigatório	<code>required</code>	■

app/models/Livro.php — método validar()

```
public function validar(array $d): array {
    $erros = [];
    if (empty(trim($d['titulo'] ?? '')))
        $erros[] = 'Título é obrigatório.';
    if (strlen(trim($d['titulo'] ?? '')) > 200)
        $erros[] = 'Título deve ter no máximo 200 caracteres.';
    if (!empty($d['ano']) && ($d['ano'] < 1000 || $d['ano'] > 2099))
        $erros[] = 'Ano inválido.';
    if (empty($d['qtd_total']) || (int)$d['qtd_total'] < 1)
        $erros[] = 'Quantidade deve ser no mínimo 1.';
    return $erros;
}
```

6. Mensagens de Sucesso e Erro

Após cada operação (criar, editar, remover), uma **flash message** é armazenada na sessão e exibida no topo da próxima página, com cores distintas para sucesso (verde) e erro (vermelho).

app/helpers.php — flash messages

```
// Armazena mensagem na sessão
function setFlash(string $key, string $msg): void {
    $_SESSION['flash'][$key] = $msg;
}

// Retorna e limpa a mensagem
function flash(string $key): ?string {
    if (isset($_SESSION['flash'][$key])) {
        $msg = $_SESSION['flash'][$key];
        unset($_SESSION['flash'][$key]);
        return $msg;
    }
    return null;
}

// Exemplo de uso no controller:
setFlash('sucesso', '■ Livro atualizado com sucesso!');
setFlash('erro', '■ Erro ao remover livro.');
```

7. CRUD Completo por Módulo

Livros

- Create — cadastro com upload de capa
- Read — listagem e visualização detalhada
- Update — edição com troca de capa
- Delete — remoção com exclusão do arquivo

Autores

- Create — cadastro com validação
- Read — listagem completa
- Update — edição de nome e nacionalidade
- Delete — remoção com confirmação

Categorias

- Create — cadastro com validação
- Read — listagem completa
- Update — edição de nome e descrição
- Delete — remoção com confirmação

8. Como Executar no XAMPP

1

Extraia **sistema-biblioteca-EP4.zip** em C:\xampp\htdocs\sistema-biblioteca

2

Inicie **Apache** e **MySQL** no painel do XAMPP

3	phpMyAdmin → aba SQL → cole o conteúdo de banco.sql → Executar
4	Acesse: http://localhost/sistema-biblioteca/public
5	Login com qualquer dado — autenticação real será implementada na EP5
6	Teste o CRUD completo: cadastrar, editar e remover livros, autores e categorias

9. Próximas Entregas

Entrega	Tema	Conteúdo	
EP5 — Sem. 12	Autenticação	Login real bcrypt, sessões, perfis e proteção de rotas	■
Final — Sem. 16	Projeto Final	Deploy, vídeo demonstrativo e documentação completa	■